Containerized Monitoring & Alerting System

System Run Sheet & Quick-Reference Guide • Docker Compose, Local Run, Locust Demos & ML Dashboard

1. System Services & Access URLs

Service	Container	Port	Access URL	Key Role
React Frontend	react_frontend	**5173**	http://localhost:5173	ML Intelligence, Live Input Graph & Locust Demo Runner
Flask App	flask_app	**5000**	http://localhost:5000	Instrumented REST API, Prometheus /metrics & Demo API
ML Service	ml_service	**5001**	http://localhost:5001	Isolation Forest Anomaly Engine (/predict & metrics)
Locust UI	locust	**8089**	http://localhost:8089	Load generator & scenario execution web UI
Prometheus	prometheus	**9090**	http://localhost:9090	TSDB scraping flask_app & ml_service, alert rules
Grafana	grafana	**3000**	http://localhost:3000	Pre-provisioned dashboards (login: admin / admin)
Alertmanager	alertmanager	**9093**	http://localhost:9093	Receives, groups and manages Prometheus alert triggers

2. Method 1: Running with Docker Compose (Recommended)

Builds, provisions, and networks all 7 services in containers via the monitoring_net bridge network:

```
# From project root:
docker compose down
docker compose up --build

# Or run in detached background mode:
docker compose up --build -d
```

Verification: Open http://localhost:5173 in your browser. The API badge will show **Online** and ML Engine will show **Active**. Open http://localhost:9090/targets to confirm both flask_app and ml_service are UP.

To Stop All Containers: docker compose down

3. Method 2: Running Locally (Development Mode)

If developing or testing without Docker Desktop, launch each service in its own terminal:

Terminal 1: Flask Backend (5000) python app/src/app.py Listens on :5000 with metrics & demo endpoints.	**Terminal 2: ML Service (5001)** python ml_service/app.py Loads Isolation Forest model & serves /predict.
Terminal 3: React Frontend (5173) cd frontend && npm run dev Vite dev server with proxies to :5000 and :5001.	**Terminal 4: Locust (Optional Web UI)** cd load-testing\locust\scripts && locust -f locustfile.py Locust web UI at http://localhost:8089.

4. Locust 1-Minute Demonstration Scenarios

Two specialized 60-second automated scenarios generate realistic traffic patterns for live alerting demonstration:

Scenario	Traffic Profile & 3 Phases	Observed Effects
demo_latency 60 Seconds	• **0–20s:** Baseline normal traffic (/process) • **20–40s Spike:** Heavy /slow delayed requests surge (40 users) • **40–60s:** Normal recovery traffic	**Latency surges to ~2.0s** during middle spike. **Errors stay at 0%.** Demonstrates pure latency degradation.
demo_errors 60 Seconds	• **0–20s:** Baseline normal traffic (/process) • **20–40s Spike:** Both /slow AND intentional 500 /error requests surge • **40–60s:** Normal recovery traffic	**Latency surges to ~2.0s.** **Errors spike to ~35–45%.** Triggers High Risk ML state & critical alerts.

How to Trigger the Demos:

Option A: One-Click from Frontend (Recommended)

Open http://localhost:5173 → In the **Interactive Locust Demo Scenarios** card, click either:

- **Run Latency Demo** (triggers demo_latency)
- **Run Latency + Errors Demo** (triggers demo_errors)

The UI displays an animated **60s countdown**, progress bar, real-time KPI metrics, and streams data directly into the Live Metrics Graph.

Option B: Run via PowerShell CLI

```
cd load-testing\locust\scripts
.\collect_dataset.ps1 -Scenario demo_latency
# OR
.\collect_dataset.ps1 -Scenario demo_errors
```

Option C: Run via REST API

```
# Trigger demo:
curl -X POST http://localhost:5000/api/demos/start -H "Content-Type: application/json" -d '{"scenario": "demo
_latency"}'

# Poll live status:
curl http://localhost:5000/api/demos/active
```

Concurrency Protection: Only 1 demo can run at a time. Trying to start a second demo returns an HTTP 409 Conflict error stating the active demo name and remaining seconds.

5. Frontend Features & Verification Checklist

A. Live Input Metrics Graph (Traffic, Latency, Errors):

- Renders real-time SVG curves for Traffic (req/s), Latency (s), and Errors (err/s or %).
- Hover over any point on the chart to view the exact timestamp and values.
- Toggle series pills to isolate or compare individual metrics.

B. Anomaly Simulator & Custom Input Evaluation:

- Adjust sliders for Request Rate, Error Rate, Latency, or Concurrency.
- Click **Evaluate Custom Inputs** → Status immediately updates to High Risk, Medium Risk, or Normal.
- Click **Reset Live** to restore real-time background metrics polling.

C. API Offline Red Banner Test:

- Stop the Flask backend: docker stop flask_app (or press Ctrl+C in Terminal 1).
- The Status Banner turns **RED** with **Status: App is Down**, an OFFLINE badge, and clear downtime alerts.
- Restart the backend → The card automatically recovers to green/normal.